

Kalshi Crypto Trading Bot — Technical Whitepaper

Gabriel Kagan

March 2026

Contents

Part 1: Executive Summary	3
What It Does	3
Market Opportunity	3
Strategy in Plain English	3
Key Differentiators	3
Part 2: System Architecture	4
Data Pipeline	4
Component Overview	4
Data Sources	6
Part 3: Technical Deep-Dive	7
3.1 Volatility Engine	7
Realized Kernel (Barndorff-Nielsen 2008)	7
Time-Varying RK Weights	7
Mincer-Zarnowitz R^2 -Weighted Blending	8
EGARCH(1,1) with Student-t Innovations	8
Adaptive Jump Detection	8
DVOL Integration	8
Cross-Asset Beta	9
3.2 Probability Model	9
Step 1: Z-Score Computation	9
Step 2: NIG CDF (Per-Asset Fitted)	9
Step 3: Data-Driven Calibration	9
Step 4: Dynamic Probability Cap	10
Step 5: Market-Price Blending	10
Sanity Checks	10
3.3 Edge Detection	11
Fee Formula	11
Fee-Adjusted Edge	11
3.4 Kalshi Order Flow (Shadow Mode)	12
3.5 S&P 500 Intraday Engine (Shadow Mode)	12
Data Sources	12
Intraday Seasonal Filter	12
EGARCH Adaptation for Equities	12

Configuration	12
3.6 Weather Temperature Engine (Shadow Mode)	13
Cities and Series	13
Ensemble Probability Model	13
Bias Correction	14
Configuration	14
3.7 Sports Comeback Engine (Shadow Mode)	14
Supported Leagues	14
Data Sources	14
Bayesian Comeback Model	15
Tennis Support	15
Entry Criteria	16
Game Lifecycle	16
3.8 Execution Strategy	16
Direct Taker Threshold	16
Three-Tier Post-Only Handler	16
Time-Based Escalation	17
Maker-to-Taker Conversion	17
Partial Fill Handling	17
UUID Persistence	17
Order Lifecycle Tracking	17
3.9 Position Sizing	18
Edge-Tiered Sizing	18
Drawdown Scaling	18
3.10 AI Analyst System	18
Part 4: Risk Management	19
Position Sizing Controls	19
Market Selection Controls	19
Model Sanity Controls	19
Execution Controls	19
Per-Window Correlation Controls (Hourly/SPX)	20
Part 5: Performance	20
Live Trading Results	20
Markets	20
Crypto 15-Minute (Live Trading)	20
Crypto Hourly (Observation Mode)	20
S&P 500 Intraday (Shadow Mode)	20
Weather Temperature (Shadow Mode)	20
Sports Outcomes (Shadow Mode)	21
Part 6: Infrastructure	21
Deployment	21
Data Persistence	21
SQLite (state.db)	21
JSONL Journals	21
Firebase Real-Time Dashboard	22

Part 1: Executive Summary

What It Does

This system is an automated trading platform for **Kalshi**, a CFTC-regulated prediction market exchange. It began with **15-minute cryptocurrency price threshold contracts** and has expanded to cover **five distinct market verticals**: crypto (15M + hourly), S&P 500 intraday, daily weather temperature, and live sports outcomes — each with domain-specific models running in shadow or observation mode alongside the live crypto engine.

The bot monitors real-time data from multiple sources per vertical, estimates outcome probabilities using domain-specific models (EGARCH volatility for crypto/SPX, NWP ensemble forecasts for weather, Bayesian comeback likelihood for sports), and trades when it identifies a statistical edge over the market price.

Market Opportunity

Kalshi lists 15-minute crypto contracts around the clock. Each window produces fresh contracts for four assets at multiple strike prices, creating hundreds of tradeable markets per day. Because these are short-duration, binary-outcome instruments, mispricing tends to be small but frequent — an ideal environment for systematic, model-driven trading.

Beyond crypto, the platform monitors four additional verticals in shadow/observation mode: S&P 500 intraday markets (EGARCH + VIX integration), daily weather temperature markets across 19 US cities (82-member NWP ensemble), live sports outcomes across 28 leagues including tennis (Bayesian comeback model), and hourly crypto markets (collecting calibration data). Each vertical uses domain-specific models while sharing the common edge detection, sizing, and execution infrastructure.

Strategy in Plain English

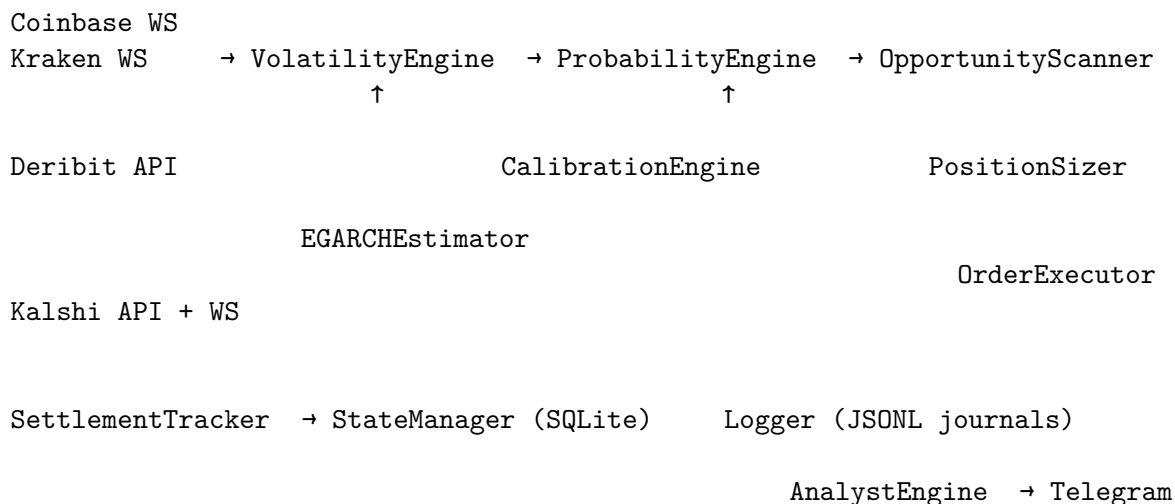
1. **Observe** — Continuously stream spot prices from Coinbase and Kraken. Fetch implied volatility from Deribit. Monitor Kalshi’s own orderbook via WebSocket.
2. **Estimate** — For every active market, compute the probability that the asset stays above its threshold using EGARCH-conditioned volatility with fat-tailed NIG distributions fitted per asset.
3. **Filter** — Reject markets that are too uncertain, too expensive, or offer insufficient edge after fees.
4. **Size** — Use edge-tiered position sizing with automatic drawdown scaling.
5. **Execute** — Place maker (limit) orders first to minimize fees, with three-tier post_only rejection handling, time-aware taker escalation, and direct taker execution below 75 seconds.
6. **Settle** — Track outcomes via the Kalshi settlements API and log performance for continuous evaluation.

Key Differentiators

Differentiator	Description
Multi-exchange intelligence	Aggregates spot prices from Coinbase and Kraken plus derivatives signals from Deribit, detecting cross-exchange lead-lag patterns before they appear in Kalshi prices
EGARCH-conditioned volatility	Realized Kernel estimation (Barndorff-Nielsen 2008) with data-adaptive bandwidth, MZ R ² -weighted blending, and EGARCH(1,1) conditional volatility — all promoted to live trading
Per-asset NIG distributions	Normal Inverse Gaussian CDF replaces the generic Student-t, capturing both heavy tails and asymmetry specific to each cryptocurrency
Adaptive execution	Three-tier post_only rejection handler, maker-first with time-aware escalation, and direct taker below 75s (data: 7.7% maker fill rate at low STC — direct taker strictly better)
Data-driven risk controls	Edge-tiered sizing (25% max), drawdown scaling, z-score sanity checks, learned calibration, and model-market discrepancy detection
Multi-vertical expansion	Five market verticals sharing common risk infrastructure, each with domain-specific probability models

Part 2: System Architecture

Data Pipeline



Component Overview

Component	Role
KalshiClient	API communication with RSA-PSS authentication and per-second rate limiting (30 reads/sec, 30 writes/sec on Advanced tier)
KalshiFeed	WebSocket connection for real-time fills and orderbook delta streaming
Logger	Structured JSONL logging across multiple journals with fill deduplication
StateManager	SQLite-backed persistent state (WAL mode for crash resilience, <code>busy_timeout=10000</code>); tracks positions, orders, fills, settlements, and order lifecycle
CoinbaseFeed	Real-time WebSocket feed for BTC, ETH, SOL, XRP with 10,800-point price buffer (15 hours at 5-second intervals)
DeribitDVOLFetcher	Daemon thread fetching implied volatility (DVOL) index for BTC and ETH every 60 seconds
CrossExchangeFeed	WebSocket feed from Kraken for cross-exchange lead-lag detection
KalshiOrderFlowTracker	Shadow-mode Kalshi-native orderbook imbalance, depth velocity, and spread convergence signals
VolatilityEngine	Realized Kernel volatility with adaptive bandwidth (H^*), MZ R^2 -weighted blending, EGARCH(1,1) Student-t conditioning, time-varying RK weights, adaptive jump detection, and DVOL integration
EGARCHEstimator	EGARCH(1,1) with Student-t innovations (df 3.2–3.8), MLE-fitted on 10,800 samples (15 hours), refitted hourly — live (promoted from shadow)
MZTracker	Mincer-Zarnowitz R^2 regression for dynamic EGARCH blend weight estimation with EMA smoothing
ProbabilityEngine	Win probability via NIG CDF (per-asset fitted) with data-driven calibration, dynamic caps (bypassed when learned calibration active), and market-price blending
CalibrationEngine	Learns calibration from settlement outcomes: Fixed Logistic → Platt Scaling → Beta Calibration → Bayesian Linear Regression (auto-promotes as data accumulates)
PositionSizer	Edge-tiered position sizing (7 tiers) with drawdown-based scaling

Component	Role
OpportunityScanner	Multi-stage filter pipeline evaluating all markets across active 15-minute windows, hourly observation windows, SPX windows, weather markets, and sports markets
OrderExecutor	Three-tier post_only handler, maker-first limit orders with adaptive taker escalation, direct taker below 75s, WebSocket fill detection, amend-first conversion, and full order lifecycle tracking
SettlementTracker	Incremental settlement polling (30-second intervals) using the Kalshi settlements API
SPXEngine	S&P 500 intraday engine: Polygon/Finnhub price feeds, EGARCH with VIX integration, intraday seasonal deseasonalization, per-window position limits (shadow mode)
WeatherEngine	Weather temperature engine: Open-Meteo NWP ensemble (82 members per run), Gaussian probability model with per-city bias correction, 19 US cities (shadow mode)
SportsEngine	Sports comeback engine: ESPN live scores across 28 leagues (incl. ATP/WTB tennis), Bayesian LR model with conservative scaling, 30-second polling (shadow mode)
AnalystEngine	AI-powered trade analyst: Claude API for loss root-cause analysis and pattern detection, Telegram alerts for high-confidence findings
TelegramNotifier	Optional Telegram alerts for trades, settlements, and errors
MainLoop	Continuous 1-second observation loop coordinating all components

Data Sources

Source	Data	Transport	Frequency
Coinbase	Spot prices (BTC, ETH, SOL, XRP)	WebSocket	Real-time (5s snapshots, 10,800-point buffer = 15hr)
Kraken	Spot prices (cross-exchange)	WebSocket	Real-time
Deribit	Implied volatility (DVOL) for BTC/ETH	REST API	60 seconds
Polygon.io	SPX spot price	REST API	1s polling (NYSE RTH)

Source	Data	Transport	Frequency
Finnhub	SPX/SPY fallback	REST API	1s polling
CBOE	VIX implied volatility	REST API	60s polling
Open-Meteo	NWP ensemble forecasts (GFS + ECMWF)	REST API	15 minutes
ESPN	Live sports scores, clock, period	REST API	30 seconds
Kalshi	Markets, orderbooks, balance, fills, settlements	REST API + WebSocket	On-demand + real-time
Claude API	AI analyst for loss analysis and pattern detection	REST API	Per-settlement

Part 3: Technical Deep-Dive

3.1 Volatility Engine

The volatility engine produces a per-asset, per-5-second realized volatility estimate that feeds the probability model. It combines multiple techniques with data-adaptive weighting and EGARCH forward-looking conditioning.

Realized Kernel (Barndorff-Nielsen 2008)

Standard sample variance of high-frequency returns is biased by market microstructure noise (bid-ask bounce, discrete tick sizes). The Realized Kernel estimator corrects this using a kernel-weighted autocovariance function with a Parzen flat-top kernel, producing noise-robust volatility estimates from 5-second log returns.

****Adaptive bandwidth (H^*)**:** Rather than using a fixed bandwidth $H=1$, the system estimates the optimal bandwidth from the data using the noise-to-signal ratio:

$$H^* = c \times \left(\frac{\hat{\omega}^2}{IV} \right)^{2/5} \times n^{3/5}$$

where $\hat{\omega}^2$ is the estimated microstructure noise variance and IV is the integrated variance. This produces tighter estimates during calm periods and wider smoothing during noisy periods.

Time-Varying RK Weights

Multiple RK estimators at different scales are blended using time-varying weights that adapt to current market conditions, rather than fixed proportions.

Mincer-Zarnowitz R²-Weighted Blending

Rather than fixed weights, the system dynamically weights estimators based on their forecasting quality. A Mincer-Zarnowitz regression compares each estimator's forecast against realized outcomes:

$$RV_{t+1} = \alpha + \beta \times \hat{\sigma}_t + \varepsilon_t$$

The R² from this regression measures forecast quality. Weights are smoothed using an EMA ($\lambda=0.97$) to prevent whipsawing:

$$w_t = \lambda \times w_{t-1} + (1 - \lambda) \times w_{raw}$$

Below an R² threshold of 0.10, the system reverts to equal-weight blending as a fallback.

EGARCH(1,1) with Student-t Innovations

The EGARCH model captures volatility clustering and leverage effects:

$$\log(\sigma_t^2) = \omega + \alpha (|z_{t-1}| - E[|z|]) + \gamma z_{t-1} + \beta \log(\sigma_{t-1}^2)$$

Fitted with Student-t innovations (df typically 3.2–3.8 for crypto) via maximum likelihood on 10,800 samples (15 hours at 5-second intervals), refitted hourly. The EGARCH forecast is blended with the RK estimate using MZ R²-weighted blending — the system dynamically determines how much weight to give the conditional model vs. the realized estimate based on forecast quality.

Status: Live — promoted from shadow mode after extensive validation. EGARCH core vol, EGARCH blend, and MZ R²-weighted blending all drive live trading decisions.

Adaptive Jump Detection

Jumps — sudden, large price moves — invalidate smooth volatility assumptions. The system uses an adaptive threshold rather than the fixed 3 σ approach:

- **EWMA variance tracking:** Tracks running variance of 15-second subsampled returns ($\lambda=0.94$)
- **Percentile-based threshold:** Jump trigger set at the asset's own volatility distribution percentile, adapting to current regime
- **Tiered response:** Jump multiplier and cooldown duration scale with jump severity
- **Health monitoring:** Logs EWMA, percentile, threshold, and total jump count per asset

DVOL Integration

When Deribit implied volatility (DVOL) exceeds realized volatility by more than 50%, the system blends in the implied estimate using inverse-variance weighting. This respects the market's forward-looking information during regime changes while anchoring to observed data.

Cross-Asset Beta

For assets without direct DVOL data (SOL, XRP), the system estimates a cross-asset beta against BTC using a 60-return lookback window, clamped to $[0.5, 3.0]$, allowing derivative signals to propagate across correlated assets.

3.2 Probability Model

Given the blended volatility, the probability engine estimates the likelihood that an asset stays above its threshold for the remaining window duration.

Step 1: Z-Score Computation

$$z = \frac{\text{threshold} - \text{spot}}{\text{spot} \times \sigma_{\text{blended}} \times \sqrt{T/5}}$$

where T is seconds remaining and σ_{blended} is per-5-second scale. The denominator represents the expected magnitude of price movement over the remaining window.

Step 2: NIG CDF (Per-Asset Fitted)

The model uses the Normal Inverse Gaussian distribution with per-asset fitted parameters:

$$p_{\text{raw}} = 1 - F_{\text{NIG}}(z; a, b, \mu, \delta)$$

where a controls tail heaviness, b captures asymmetry (skew), μ is location, and δ is scale. NIG parameters are fitted via maximum likelihood on 7 days of 60-second returns (~10,000 samples per asset) and stored in `dist_config.json`.

Why NIG over Student-t? NIG provides two key improvements: - **Asymmetry**: The b parameter captures the empirical skew in crypto returns (e.g., BTC $b = -0.019$, slight left skew) - **Better tail fit**: KS test p-values for NIG are dramatically higher (BTC: 0.11, ETH: 0.42) compared to Student-t (effectively 0), indicating NIG genuinely captures the return distribution

The system falls back to Student-t(df=4) if NIG parameters are unavailable.

Step 3: Data-Driven Calibration

Raw probabilities are calibrated using a CalibrationEngine that learns from settlement outcomes. The engine automatically promotes to better methods as data accumulates:

Fixed Logistic (=0.85) 0 samples → Platt Scaling 200+ samples → Beta Calibration 350+ samples → Bayesian LR 50+ samples
(default fallback) (2-param logistic) (3-param, flexible) (with uncertainty)

Method	Min Samples	Description
Fixed logistic (=0.85)	0	Default fallback — compresses extreme probabilities
Platt Scaling	200	2-parameter logistic (A, B) fitted to outcomes

Method	Min Samples	Description
Beta Calibration	350	3-parameter (a, b, c) — more flexible than Platt
Bayesian Linear Regression	50	Bayesian approach with uncertainty estimation

Each promotion undergoes validation checks to prevent degradation. **15-minute data only:** Hourly settlement data is excluded from calibration training (was contaminating the 15M model at 35.5% of training data).

Step 4: Dynamic Probability Cap

A time-dependent cap adjusts confidence based on time remaining:

Time to Expiry	Cap
> 10 minutes	93%
5–10 minutes	95%
2–5 minutes	97%
1–2 minutes	98.5%
< 1 minute	99.5%

Important: When a learned calibration method is active (Platt, Beta, or BLR trained on settlement data), the dynamic cap schedule is **bypassed entirely**. A numerical safety ceiling of 99.9% is used instead. The rationale is that learned calibration already accounts for the overconfidence the cap was designed to prevent. The cap schedule only applies during startup before sufficient training data accumulates.

As expiry approaches and less can go wrong, the cap relaxes to allow higher-confidence trades in the endgame.

Step 5: Market-Price Blending

The calibrated probability is blended with the market-implied probability:

$$p_{final} = 0.60 \times p_{cal} + 0.40 \times p_{market}$$

This 60/40 blend (60% model, 40% market) was validated against a no-blend alternative: the no-blend system was +1.86 percentage points overconfident (Brier score 0.0946 vs 0.0422), and would have generated 16 trades that were net -\$53.54. The 40% market weight was subsequently tuned from 50% after data showed the model was underconfident by 0.8–2.1pp at 90%+ probabilities. The no-blend system now monitors in shadow mode.

Sanity Checks

- **Z-score limit:** If $|z| > 25$, the market is refused (all historical losses have z-scores below this threshold; the previous limit of 12 was blocking only winners)

- **Model-market discrepancy:** If $p_{cal} > 90\%$ but market price $< 75\text{¢}$, the market is refused (the model may be missing information the market has)

3.3 Edge Detection

A trade requires positive expected value after accounting for fees.

Fee Formula

Kalshi charges fees using a variance-based formula:

$$\begin{aligned}\text{taker fee} &= \lceil 0.07 \times C \times P \times (1 - P) \rceil \text{ cents} \\ \text{maker fee} &= \lceil 0.0175 \times C \times P \times (1 - P) \rceil \text{ cents}\end{aligned}$$

where C is the number of contracts and P is the trade price as a decimal. The ceiling is applied to the total, not per contract. SPX (“finance” category) uses half the crypto fee multiplier: 0.035 taker / 0.0175 maker.

Fee-Adjusted Edge

The scanner evaluates edge using taker fees (worst-case), so any candidate that passes the filter is profitable even if maker order is rejected:

$$\text{edge} = p_{final} - \frac{\text{best_ask}}{100} - \frac{\text{taker_fee}}{C \times 100}$$

A trade must satisfy:

$$\text{edge} \geq \text{get_min_edge}(\text{price})$$

The minimum edge is price-dependent, reflecting the higher risk of expensive contracts:

Entry Price	Min Edge
97¢+	4.0%
95–96¢	2.5%
93–94¢	1.8%
91–92¢	1.2%
89–90¢	0.9%
87–88¢	0.7%
86¢	0.5%

A flat fallback of 0.7% (MIN_EDGE_PCT) applies if the price-dependent schedule is unavailable.

3.4 Kalshi Order Flow (Shadow Mode)

The KalshiOrderFlowTracker monitors Kalshi’s own orderbook for predictive signals:

- **Imbalance:** Ratio of bid vs. ask depth — strong imbalance (>0.8 or <0.2) suggests directional pressure
- **Depth velocity:** Rate of change in total depth — draining liquidity may predict a move
- **Spread convergence:** Narrowing spread + trending depth suggests informed trading
- **Adjustments:** ± 1 to 1.5pp based on signal strength, with confidence levels based on snapshot count

Currently logging only — signals are computed but do not affect trading decisions.

3.5 S&P 500 Intraday Engine (Shadow Mode)

The SPX engine trades S&P 500 15-minute prediction markets (KXINXU series) using the same EGARCH framework adapted for equity microstructure.

Data Sources

Source	Data	Frequency
Polygon.io	SPX spot price	1s polling during RTH
Finnhub	SPX fallback + SPY→SPX conversion ($10.03\times$)	1s polling
CBOE VIX	Implied volatility index	60s polling

Intraday Seasonal Filter

SPX volatility follows a well-documented U-shaped intraday pattern (high at open/close, low mid-day). The engine deseasonalizes returns using 13 half-hour buckets (09:30–16:00 ET) with EWMA-calibrated seasonal factors, preventing systematic bias from time-of-day effects.

EGARCH Adaptation for Equities

The EGARCH(1,1) model is re-parameterized for equity-specific dynamics:

- **Leverage bounds:** $\gamma \in (-0.30, -0.05)$ — approximately $4\times$ stronger than crypto, reflecting the well-documented equity leverage effect (down moves increase volatility more than up moves)
- **VIX integration:** When VIX-implied vol diverges $>30\%$ from realized, the engine shifts 30% weight toward VIX. On startup, EGARCH is seeded from VIX to avoid a cold-start period
- **Market hours guard:** NYSE RTH 9:30–16:00 ET with DST awareness and holiday calendar. Engine skips the first 10 minutes post-open (auction noise)

Configuration

Config	Value
Status	Shadow (observation only)
Entry price range	70–99¢

Config	Value
STC window	300–1800s
Market blend	60/40 (model/market)
Max risk per trade	15%
Kelly fraction	0.25 (quarter-Kelly)
Fee multiplier (taker)	0.035 (half crypto)
Fee multiplier (maker)	0.0175
Max positions per window	2
Max risk per window	15%

3.6 Weather Temperature Engine (Shadow Mode)

The weather engine trades daily high temperature prediction markets across **19 US cities** using numerical weather prediction (NWP) ensemble forecasts.

Cities and Series

City	Series Ticker
New York	KXHIGNY
Chicago	KXHIGNCHI
Miami	KXHIGNMIA
Denver	KXHIGNDEN
Los Angeles	KXHIGNLAX
Austin	KXHIGNHAUS
Atlanta	KXHIGNTATL
San Francisco	KXHIGNTSFO
Dallas	KXHIGNTDAL
Phoenix	KXHIGNTPHX
Philadelphia	KXHIGNPHIL
Minneapolis	KXHIGNTMIN
Seattle	KXHIGNTSEA
Houston	KXHIGNTHOU
Boston	KXHIGNTBOS
Las Vegas	KXHIGNTLV
Oklahoma City	KXHIGNTOKC
Washington DC	KXHIGNTDC
New Orleans	KXHIGNTNOLA

Each city has Kalshi bracket and threshold markets settling daily based on the observed high temperature.

Ensemble Probability Model

The engine queries the Open-Meteo API for two independent NWP ensemble systems:

Model	Members	Resolution	Provider
GFS Seamless	31	~13 km	NOAA
ECMWF IFS 0.25°	51	~25 km	ECMWF
HRRR (deterministic)	1	<15 km	NOAA

The 82 ensemble members (31 GFS + 51 ECMWF) provide a distribution of possible temperature outcomes. The engine fits a Gaussian (μ, σ) to the combined ensemble and computes:

- **Bracket markets:** $P(\text{lower} < T < \text{upper})$ via CDF difference
- **Threshold markets:** $P(T > \text{threshold})$ or $P(T < \text{threshold})$ via tail probability

Bias Correction

An EWMA bias tracker ($\lambda = 0.90$, 7-day half-life) maintains per-city forecast error history. After each day’s actual temperature is observed, the engine updates $\text{bias}_{\text{city}} = \lambda \times \text{bias}_{\text{prev}} + (1 - \lambda) \times (\text{actual} - \text{forecast})$ and shifts the ensemble mean accordingly.

Configuration

Config	Value
Status	Shadow (observation only)
Entry price range	10–99¢
Settle window	Daily (min 1hr before close)
Market blend	80/20 (model/market) — ensemble is primary signal
Max risk per trade	10%
Kelly fraction	0.25 (quarter-Kelly)
Poll interval	15 minutes (weather changes slowly)
Max cities per day	19 (all enabled for data collection)

3.7 Sports Comeback Engine (Shadow Mode)

The sports engine monitors live games across 28 leagues for Bayesian comeback signals — identifying situations where a pregame favorite is trailing but statistically likely to recover.

Supported Leagues

Binary outcome (home/away): NBA, NHL, MLB, NCAAB, NCAAF, NFL, WNBA, UFC, ATP Tennis, WTA Tennis, plus esports (CS:GO, LoL, Valorant — Kalshi price monitoring only)

Three-way outcome (home/draw/away): EPL, Bundesliga, La Liga, Serie A, UCL, Ligue 1, MLS, Liga MX, Europa League, Conference League, Super Lig, Eredivisie, World Cup, FIFA Friendlies, AFC Asian Cup

Data Sources

Source	Data	Coverage
ESPN API	Live scores, clock, period, red cards	Leagues with live scoreboards
Kalshi API	Game-level market prices	28 series (2 markets/binary game, 3/three-way)

Bayesian Comeback Model

The model computes posterior comeback probability using a lookup-table of empirically calibrated likelihood ratios:

$$P(\text{comeback} \mid \text{data}) = \frac{LR \times P(\text{prior})}{LR \times P(\text{prior}) + (1 - P(\text{prior}))}$$

Likelihood ratio table keys: (d, t, s) where: - d = deficit bucket (binary: small/medium/large/blowout; three-way: 1-goal/2-goal/3+) - t = time remaining bucket ($>75\%$, $50-75\%$, $25-50\%$, $<25\%$) - s = pregame strength bucket (strong favorite 75% , moderate $65-75\%$, slight $55-65\%$)

Conservative scaling: LR values are compressed 80% toward neutral ($LR_{\text{scaled}} = 1.0 + (LR_{\text{raw}} - 1.0) \times 0.2$) to prevent overconfident signals. This aggressive compression was chosen because the LR tables are based on general historical comeback rates that may not transfer to Kalshi-specific market dynamics.

Model-market safety cap: Signals are rejected when the model posterior exceeds the market price by more than 80 percentage points, allowing wide-gap data collection for model calibration. This threshold will be tightened once sufficient calibration data accumulates.

Tennis Support

Tennis (ATP and WTA) uses a specialized parsing pipeline due to structural differences from team sports:

- **ESPN structure:** Tennis events are organized as tournament $\rightarrow$ groupings $\rightarrow$ competitions, with an extra nesting level compared to team sports
- **Gender filtering:** ESPN returns all genders at a tournament endpoint; the engine filters to “Men’s Singles” for ATP and “Women’s Singles” for WTA to prevent cross-gender contamination
- **Player codes:** ESPN has no 3-letter abbreviations for tennis players. Codes are derived from player last names (first 3 alpha characters of concatenated surname parts): “Jannik Sinner” $\rightarrow$ “SIN”, “Alex de Minaur” $\rightarrow$ “DEM”, “Christopher O’Connell” $\rightarrow$ “OCO”
- **Scoring:** Sets won (count of `linescores` entries with `winner=True`) rather than cumulative points
- **Time remaining:** Estimated from sets completed + games played in current set (no clock in tennis)
- **Deficit classifier:** 1 set down = “small” (common comeback), 2+ sets down = “large” (rare, best-of-5 only)

Entry Criteria

Current entry criteria are relaxed for maximum data collection in observation mode:

Criteria	Binary	Three-way
Min pregame favorite prob	50%	50%
Max Kalshi favorite price	95¢	95¢
Min time remaining	10%	10%
Max deficit bucket	Blowout	3 goals
Signal dedup	One signal per game	One signal per game

These thresholds are intentionally permissive to capture wide-gap and late-game data for model calibration. They will be tightened before any promotion to live trading.

Game Lifecycle

1. **Pregame capture:** Record Kalshi prices for each team before game starts
2. **Live evaluation:** Poll ESPN scores every 30 seconds, compute LR on each score change
3. **Signal dedup:** One signal per game to prevent correlated exposure
4. **Settlement:** After game ends, record final outcome, closing prices, and simulated P&L

3.8 Execution Strategy

The executor uses a maker-first approach with three-tier post_only rejection handling, time-aware escalation, and direct taker execution for late-window entries.

Direct Taker Threshold

When less than **75 seconds** remain before settlement, the system skips the maker order entirely and submits a direct IOC (immediate-or-cancel) taker order.

Data justification: Maker fill rate was only 7.7% (1/13 candidates) at 0–60s STC. Twelve missed candidates were all winners (~\$49 net missed profit). Threshold raised from 60s → 75s. Edge and liquidity checks still apply.

Three-Tier Post-Only Handler

When a `post_only=True` maker order is rejected (the order would cross the spread rather than rest on the book), the system escalates through three tiers:

Tier	Trigger	Action
Tier 1: Normal maker	0–1 rejections	Standard maker order, 1–2¢ below fair value
Tier 2: Degraded maker	2 rejections	Same offset + 1¢ additional discount. If price drops below 86¢ floor, skipped.

Tier	Trigger	Action
Tier 3: Taker IOC	3+ rejections	Edge re-verified with actual taker fees → IOC order if still profitable

Rejection counts expire after 30 seconds and are per-ticker (unique per market window).

Time-Based Escalation

For orders that are successfully placed but sit unfilled:

Urgency	Time to Close	Maker Wait
Low	180s	15s
Medium	120–180s	7s (86% of fills happen within 7s)
High	60–120s	5s

Maker-to-Taker Conversion

1. Place maker order with `post_only=True` (guarantees maker fees, 4× cheaper)
2. Monitor for fills via Kalshi WebSocket (zero API cost) with REST polling fallback
3. Poll queue position every ~5s for queue-aware escalation timing
4. If timeout reached without fill:
 - Attempt `amend_order()` to convert to taker price in-place (avoids cancel+replace race)
 - If amend fails, fall back to cancel + IOC (`time_in_force="immediate_or_cancel"`) taker order
 - Re-validate price still in [86¢, 99¢] before taker submission

Taker execution data: Taker trades show 14W/0L (100% win rate) across all STC zones. The previous maker-only threshold of 90s was removed after this data demonstrated taker execution is profitable at all time horizons.

Partial Fill Handling

Orders may partially fill (e.g., 3 of 13 contracts). The execution engine tracks `filled_so_far` cumulatively and keeps the order active until fully filled or escalated. REST fill detection uses a `_seen_fill_ids` set to prevent double-counting across consecutive polls and against WebSocket fills. Escalation to taker subtracts partial fills from the IOC count to prevent position doubling.

UUID Persistence

Each order gets a `client_order_id` (UUID4) written to SQLite before API submission. This ensures crash recovery — if the bot restarts mid-order, it can reconcile using the persisted UUID.

Order Lifecycle Tracking

Every evaluated opportunity that reaches candidate status gets full order lifecycle tracking:

- `order_id`: Kalshi’s assigned order identifier

- **order_submitted_at**: Timestamp of API submission
- **order_outcome**: Final disposition — `filled`, `unfilled`, `canceled`, or `partial_fill`

This enables post-hoc analysis of execution quality, fill rates by STC zone, and maker vs. taker performance comparison.

3.9 Position Sizing

Edge-Tiered Sizing

Position sizing is tiered by fee-adjusted edge, with higher-conviction trades receiving larger allocations:

Fee-Adjusted Edge	Risk Fraction
4%	25% of bankroll
2.5%	20% of bankroll
1.8%	15% of bankroll
1.2%	10% of bankroll
0.9%	7% of bankroll
0.7%	5% of bankroll
0.5%	3% of bankroll

Safety ceiling: max 25% of bankroll at risk per trade.

Drawdown Scaling

Balance vs. Starting	Sizing Adjustment
85%	Full sizing
75–85%	Half sizing
65–75%	Quarter sizing
< 65%	Halt trading

This creates a geometric de-risking curve that preserves capital during losing streaks.

3.10 AI Analyst System

The analyst engine (`analyst.py`) uses the Claude API to provide automated post-trade analysis:

- **Loss root-cause analysis**: After every losing trade, the analyst examines market conditions, volatility regime, entry timing, and model state to identify the cause
- **Pattern detection**: Identifies recurring loss patterns across assets, time windows, and market conditions
- **Telegram alerts**: High-confidence findings are pushed to Telegram for real-time operator awareness
- **Non-interfering**: The analyst runs asynchronously and never affects trading decisions — it is purely diagnostic

Part 4: Risk Management

Position Sizing Controls

- **Edge-tiered sizing:** Position size scales with conviction — 25% max at 4%+ edge, down to 3% at 0.5% edge
- **Drawdown scaling:** Size halved below 85% of starting balance, quartered below 75%, trading halted below 65%
- **Hard limits:** Maximum risk per trade capped at 25% of bankroll

Market Selection Controls

- **Multi-asset capable:** Can trade multiple assets per 15-minute window
- **Price range guardrails:** Only trade contracts priced 86–99¢ — below 86¢ has historically poor win rates; above 99¢ offers insufficient reward
- **Price-dependent edge threshold:** Fee-adjusted edge must exceed a price-dependent minimum (0.5% at 86¢ up to 4.0% at 97¢+) after taker fees (worst-case)
- **Scanner uses taker fees:** Every candidate is profitable even if forced to taker execution

Model Sanity Controls

- **Z-score limit:** Refuse markets where $|z| > 25$ (validated against settlement data: 82 tradeable z-score rejections above 12 were all winners, leading to the raise from 12 $\rightarrow$ 25)
- **Model-market discrepancy:** If the model estimates $>90\%$ probability but the market prices below 75¢, refuse (the model may be missing material information)
- **Dynamic probability cap:** Time-dependent ceiling (93–99.5%) prevents overconfidence during startup; bypassed (99.9% ceiling) once learned calibration is active
- **Data-driven calibration:** CalibrationEngine learns from settlement outcomes, replacing fixed assumptions with empirical mappings
- **Market-price blending:** 60/40 blend (60% model, 40% market) anchors estimates and prevents systematic overconfidence

Execution Controls

- **Three-tier post_only handler:** Escalates from normal maker $\rightarrow$ degraded maker $\rightarrow$ taker IOC after repeated rejections, with edge re-verification at each tier
- **Maker-first with post_only:** Guarantees maker fee tier (75% cheaper), rejected if it would cross the spread
- **Direct taker below 75 seconds:** Below 75s STC, maker orders are skipped entirely (7.7% fill rate) — direct IOC taker submitted with full edge/liquidity validation
- **WebSocket fill detection:** Zero-cost fill monitoring via Kalshi WebSocket, with REST polling fallback
- **Amend-first escalation:** Uses `amend_order()` API to convert maker $\rightarrow$ taker in-place, avoiding cancel+replace race conditions
- **IOC taker orders:** Taker escalation uses `time_in_force="immediate_or_cancel"` to prevent stale resting orders
- **Partial fill tracking:** Escalation subtracts filled contracts to prevent position doubling
- **Price re-validation:** After maker timeout, the system re-fetches the orderbook and re-validates the price range before submitting a taker order

- **UUID persistence:** Order IDs written to disk before API submission, enabling crash recovery without duplicate orders
- **Rejection expiry:** Post_only rejection counts expire after 30 seconds, preventing stale state from affecting future windows

Per-Window Correlation Controls (Hourly/SPX)

- **Max positions per window:** 2 (limits correlated multi-strike exposure)
- **Max risk per window:** 15% of bankroll (prevents simultaneous multi-asset blowups)
- **Quarter-Kelly sizing:** 0.25 Kelly fraction for non-15M verticals (44% of growth rate, ~3% halving probability)

Part 5: Performance

Live Trading Results

Metric	Value
Status	Live trading since February 22, 2026
Settled trades	258
Win rate	88.8% (229W / 29L)
Assets	BTC, ETH, SOL, XRP

Markets

Crypto 15-Minute (Live Trading)

Binary contracts settling every 15 minutes. Series: KXBTC15M, KXETH15M, KXSOL15M, KXXRP15M.

Crypto Hourly (Observation Mode)

75 strikes per event, settling every hour. Currently collecting calibration data only — no live trading. Was briefly promoted to live trading (Feb 27–28) but reverted after -\$97 overnight disaster from calibration overconfidence and correlated multi-strike exposure. Series: KXBTCD, KXETHD, KXSOLD, KXXRPD.

S&P 500 Intraday (Shadow Mode)

15-minute binary contracts on the S&P 500 during NYSE regular trading hours. Series: KX-INXU. Uses equity-adapted EGARCH with VIX integration and intraday seasonal adjustment. Per-window limits: max 2 positions, 15% risk cap.

Weather Temperature (Shadow Mode)

Daily high temperature markets across 19 US cities. Bracket and threshold contracts settling based on the observed daily high. Probability from 82-member NWP ensemble (GFS + ECMWF).

Sports Outcomes (Shadow Mode)

Live game outcome markets across 28 leagues including NBA, NHL, MLB, NFL, EPL, ATP/WT A Tennis, and more. Bayesian comeback model identifies edge when pregame favorites trail in-game. Binary and three-way (soccer draw) market types.

Part 6: Infrastructure

Deployment

Component	Detail
Host	DigitalOcean droplet (Ubuntu 24.04), 1 vCPU / 2GB RAM / 48GB disk
Runtime	Python 3, virtualenv
Process manager	systemd (kalshi-bot service) — auto-restarts on crash
Startup	start.sh activates venv, sources .env, launches bot.py
Auto-deploy	Push to main → GitHub Action → SSH → pull → syntax-check → restart

Data Persistence

SQLite (state.db)

The primary state store uses SQLite in WAL (Write-Ahead Logging) mode for crash resilience. All connections use busy_timeout=10000 to handle concurrent access from multiple threads (bot main loop, Firebase push, sports engine).

Table	Purpose
positions	Active positions (ticker, asset, side, count, avg price)
pending_orders	Orders awaiting fill (with UUID client_order_id)
settled_trades	Completed trades with P&L, product_type, and execution metadata
rejected_opportunities	Markets rejected with reason, model state, and product_type
evaluated_opportunities	Every market evaluation with filter stage, order lifecycle tracking (order_id, order_submitted_at, order_outcome), and available_balance_cents

JSONL Journals

Append-only journal files provide a complete audit trail:

Journal	Contents
<code>opportunity_journal.jsonl</code>	Filter stage tracking for every market evaluation
<code>scan_journal.jsonl</code>	Per-tick scan summaries (~330MB/day)
<code>rejection_journal.jsonl</code>	Settlement outcomes for rejected opportunities
<code>fill_model_journal.jsonl</code>	Maker order lifecycle data for ML fill prediction

Journal rotation: A daily cron job (4 AM UTC) runs `rotate_journals.sh` using a copytruncate pattern — journals are compressed to `journal_archives/` with gzip and 30-day retention. The bot uses open/close per write, so rotation is safe without process interruption.

Firestore Real-Time Dashboard

A Firestore integration provides a live web dashboard showing:

- Current positions and P&L
- Active market evaluations
- Volatility regime indicators and EGARCH/NIG parameters
- Orderbook visibility for active windows
- Execution engine statistics (amend success rate, WS fill ratio, post_only rejection counts, taker escalation counts)
- Calibration diagnostics
- Hourly observation stats
- SPX shadow data
- Weather observation data
- Sports shadow signals

Shadow Mode Features

The system supports shadow mode for experimental features — they compute and log but do not affect live trading decisions:

Feature	Status	Purpose
S&P 500 Intraday	Shadow	EGARCH + VIX vol model for SPX 15M contracts (KXINXU)
Weather Temperature	Shadow	82-member NWP ensemble for daily high temperature markets (19 cities)
Sports Comeback	Shadow	Bayesian LR comeback model across 28 leagues (incl. ATP/WTB tennis)
Hourly Crypto	Observation	Collecting calibration data for hourly markets (75 strikes/event)

Feature	Status	Purpose
Kalshi Order Flow	Shadow	Orderbook imbalance, depth velocity, spread convergence signals
Sigmoid QLIKE	Shadow	Alternative EGARCH weight via QLIKE improvement ratio
Shadow Cal Pipeline	Shadow	No-blend calibration monitoring (was promoted, caused +1.86pp overconfidence)
Dip Addon	Shadow	Buy more when ask dips 3¢ below entry after fill (50% addon size, 35% total risk cap)

Promoted features (driving live behavior): - **EGARCH core vol** — EGARCH(1,1) with Student-t innovations - **EGARCH blend** — MZ R²-weighted blending of EGARCH vs RK - **Time-varying RK weights** — adaptive multi-scale RK blending - **Adaptive jump detection** — percentile-based thresholds per asset - **Adaptive RK bandwidth** — data-driven H* selection - **Temperature calibration** — competes in hourly Brier tournament

Last updated: 2026-03-06T23:11:09Z